



MÓDULO 5 · T14

Persistencia y Arquitectura: AsyncStorage



Persistencia local con AsyncStorage

Carlos Rodríguez Contreras · GitHub: [karrocon](#)

¿Qué aprenderás hoy?

En esta sesión aprenderás a persistir datos localmente en React Native usando AsyncStorage. Al terminar, serás capaz de guardar preferencias del usuario y cachear datos de API para mejorar el rendimiento de tu aplicación.

1

Instalar AsyncStorage

Añadir `@react-native-async-storage/async-storage` al proyecto con Expo.

2

Wrappers genéricos

Implementar `saveJSON` / `loadJSON` reutilizables para serializar objetos.

3

Persistir preferencias

Guardar y restaurar ajustes del usuario entre sesiones con un hook personalizado.

4

Caché de API

Usar AsyncStorage como caché para reducir el tiempo de carga de datos remotos.

¿Qué es AsyncStorage?

AsyncStorage es el equivalente a `localStorage` del navegador en React Native, pero con características específicas para aplicaciones móviles. Es la solución estándar para persistir datos simples de forma local en el dispositivo.

Asíncrono

Siempre devuelve `Promises` – debes usar `await` o `.then()`.

Persistente

Los datos sobreviven entre cierres de la app y reinicios del dispositivo.

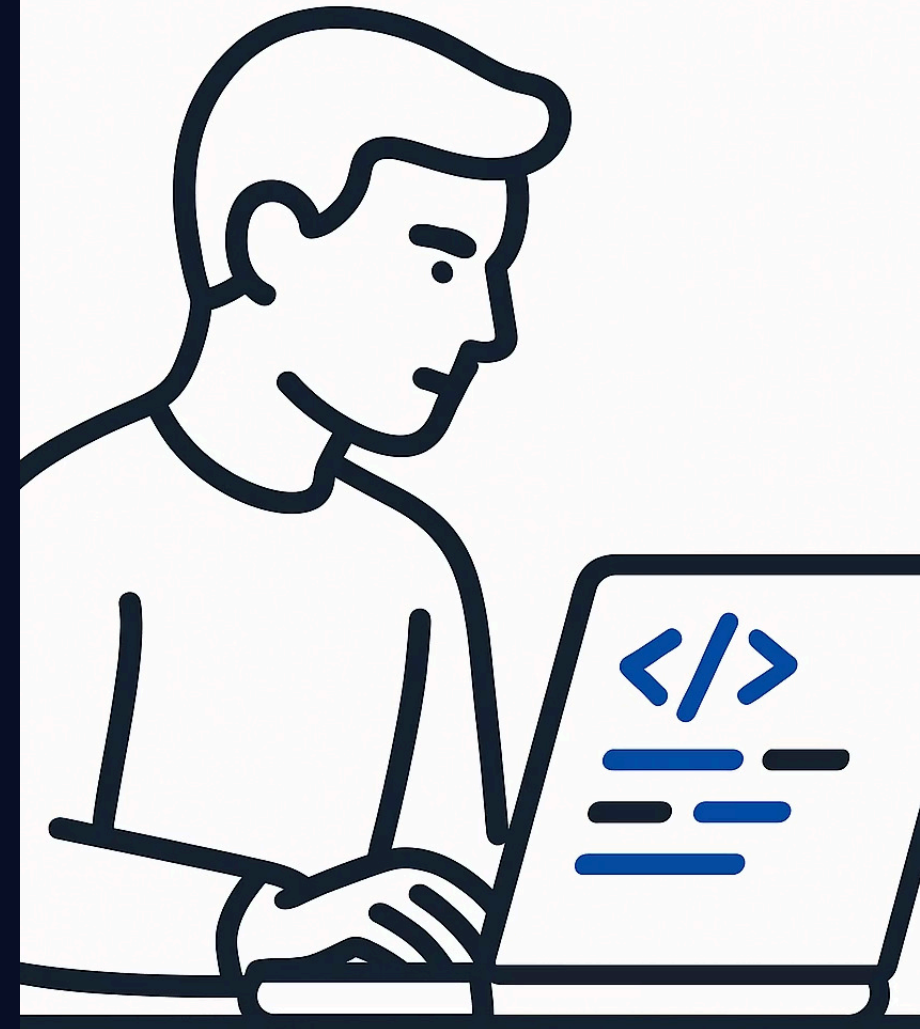
Solo strings

Únicamente almacena cadenas de texto – hay que serializar objetos a JSON.

Capacidad limitada

Capacidad aproximada de ~6 MB en iOS – no apta para datos grandes.

❗ **AsyncStorage** no debe usarse para datos sensibles (contraseñas, tokens). Para eso existe `expo-secure-store`.



Instalación y API básica

Instalación con Expo

```
npx expo install @react-native-async-storage/async-storage
```

Usa siempre `expo install` en lugar de `npm install` para garantizar la versión compatible con tu SDK de Expo.

- ✓ Con Expo Managed Workflow no es necesario ningún paso nativo adicional.

Operaciones básicas

```
import AsyncStorage from '@react-native-async-storage/async-storage';

// Guardar
await AsyncStorage.setItem('clave', 'valor');

// Leer (null si no existe)
const valor = await AsyncStorage.getItem('clave');

// Eliminar
await AsyncStorage.removeItem('clave');

// Limpiar todo el storage
await AsyncStorage.clear();
```

Wrappers genéricos para JSON

Dado que AsyncStorage solo almacena strings, es buena práctica encapsular la serialización y deserialización en dos funciones genéricas con TypeScript. Estos wrappers son exactamente los de `PokeApp/src/Utils/storage.ts`.

```
export async function saveJSON<T>(  
  key: string,  
  value: T  
) : Promise<void> {  
  await AsyncStorage.setItem(key, JSON.stringify(value));  
}
```

```
export async function loadJSON<T>(  
  key: string  
) : Promise<T | null> {  
  const raw = await AsyncStorage.getItem(key);  
  return raw ? (JSON.parse(raw) as T) : null;  
}
```

❗ El genérico `<T>` permite que TypeScript infiera el tipo del valor guardado o cargado, manteniendo el tipado estricto en toda la app.

Persistir preferencias del usuario

Con `saveJSON` / `loadJSON` podemos construir un hook personalizado que carga los ajustes guardados al montar el componente y los persiste automáticamente cada vez que cambian.

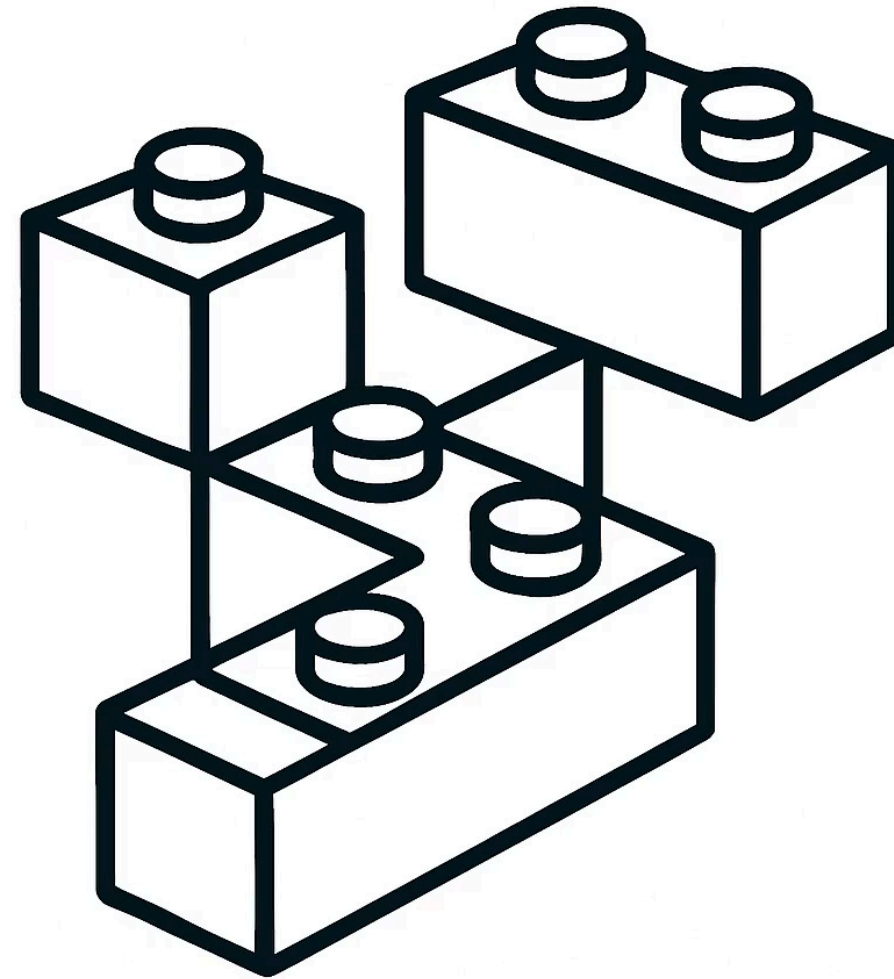
```
const SETTINGS_KEY = 'user_settings';

export function useAjustes() {
  const [settings, setSettings] =
    useState(DEFAULT_SETTINGS);

  // Carga al montar
  useEffect(() => {
    loadJSON<Settings>(SETTINGS_KEY).then(saved => {
      if (saved) setSettings(saved);
    });
  }, []);

  // Persiste cualquier cambio
  const update = (partial: Partial<Settings>) => {
    setSettings(prev => {
      const next = { ...prev, ...partial };
      saveJSON(SETTINGS_KEY, next);
      return next;
    });
  };

  return { settings, update };
}
```



Caché de API con AsyncStorage

AsyncStorage permite cachear los datos de una API para que la segunda apertura sea casi instantánea. Si existe caché válida, se muestra al usuario sin tocar la red; solo si no hay caché se ejecuta el fetch.

```
// En usePokemonsContext — carga rápida desde caché  
const cached = await loadJSON<Pokemon[]>(CACHE_KEY);
```

```
if (cached && cached.length > 0) {  
  setPokemons(cached);  
  setIsLoading(false);  
  return; // ← evita el fetch a la API  
}
```

```
// Solo si no hay caché, fetcha y guarda  
const all = await fetchAllPokemons();  
await saveJSON(CACHE_KEY, all);
```

✓ Reduce el tiempo de carga de ~30 s a ~0.1 s en la segunda apertura. 🚀

Tour starter/storage.ts

Este es el fichero que encontrarás en la carpeta `starter/`. Contiene las dos funciones marcadas como `TODO` que deberás implementar durante el ejercicio. Por ahora solo lanzan un error – tu misión es hacerlas funcionales.

```
export async function saveJSON<T>(  
  key: string,  
  value: T  
): Promise<void> {  
  throw new Error('TODO: implementar saveJSON');  
}
```

```
export async function loadJSON<T>(  
  key: string  
): Promise<T | null> {  
  throw new Error('TODO: implementar loadJSON');  
}
```

⚠ No modifiques los tipos ni las firmas de las funciones – concéntrate únicamente en la implementación de cada `TODO`.

✓ SOLUCIÓN

Solución — solucion/storage.ts

La solución completa implementa saveJSON con JSON.stringify y loadJSON con JSON.parse, manejando correctamente el caso null. Compara tu implementación con esta referencia.

```
export async function saveJSON<T>(  
  key: string,  
  value: T  
): Promise<void> {  
  await AsyncStorage.setItem(key, JSON.stringify(value));  
}  
  
export async function loadJSON<T>(  
  key: string  
): Promise<T | null> {  
  const raw = await AsyncStorage.getItem(key);  
  return raw ? (JSON.parse(raw) as T) : null;  
}  
  
export async function removeKey(key: string): Promise<void> {  
  await AsyncStorage.removeItem(key);  
}  
  
export async function clearAll(): Promise<void> {  
  await AsyncStorage.clear();  
}
```

✓ Si el contador persiste entre reinicios de la app sin errores de TypeScript, ¡lo has conseguido! 🎉

Ejercicio T14

Pon en práctica todo lo aprendido. Abre el proyecto en tu editor y sigue los pasos en orden – cada uno construye sobre el anterior. El bonus es opcional pero muy recomendable para afianzar el uso de AsyncStorage en un componente real.

1 Abre el fichero de inicio

Navega a `starter/storage.ts` en tu editor.

2 Implementa `saveJSON`

Usa `JSON.stringify` y `AsyncStorage.setItem`.

3 Implementa `loadJSON`

Usa `JSON.parse` y maneja el caso `null` cuando la clave no existe.

4 Implementa `removeKey` y `clearAll`

Delega en `AsyncStorage.removeItem` y `AsyncStorage.clear`.

5 Bonus: contador persistente

Usa los wrappers en un componente para persistir un contador entre reinicios de la app.

Resumen de la sesión

Estas son las funciones clave de AsyncStorage que has trabajado hoy. Dominarlas te permitirá añadir persistencia local a cualquier aplicación React Native de forma sencilla y tipada.

Función	Uso	Para qué sirve
<code>setItem(key, string)</code>	<code>await AsyncStorage.setItem(k, v)</code>	Guardar un string en el storage
<code>getItem(key)</code>	<code>await AsyncStorage.getItem(k)</code>	Leer un string (null si no existe)
<code>removeItem(key)</code>	<code>await AsyncStorage.removeItem(k)</code>	Eliminar una clave del storage
<code>clear()</code>	<code>await AsyncStorage.clear()</code>	Limpiar todo el storage
<code>saveJSON<T></code>	<code>saveJSON(key, objeto)</code>	Wrapper para guardar objetos tipados
<code>loadJSON<T></code>	<code>loadJSON<T>(key)</code>	Wrapper para leer objetos tipados

✅ ¡Enhorabuena! Ahora tienes las herramientas para persistir datos localmente, cachear respuestas de API y guardar preferencias del usuario en React Native. 🚀